

# Clean Code Notes – Args Case Study

*“First make it work, then make it right, then make it fast.” – Kent Beck*

## 1. Overview

This document explains the evolution of a command-line argument parser (Args) from messy code into clean, maintainable architecture using refactoring and SRP.

## 2. Bad Code Example (Messy Design)

```
private boolean setArgument(char argChar) throws ArgsException {
    if (isBooleanArg(argChar))
        setBooleanArg(argChar, true);
    else if (isStringArg(argChar))
        setStringArg(argChar);
    else if (isIntArg(argChar))
        setIntArg(argChar);
    else
        return false;
    return true;
}
```

Problems: - Too many conditionals (if-else chain) - Violates Open/Closed Principle - Hard to extend (adding new types breaks code) - Mixed responsibilities

## 3. Good Code Example (Clean Design)

```
private void parseArgumentCharacter(char argChar) throws ArgsException {
    ArgumentMarshaler m = marshalers.get(argChar);
    if (m == null) {
        throw new ArgsException(UNEXPECTED_ARGUMENT, argChar, null);
    } else {
        argsFound.add(argChar);
        m.set(currentArgument);
    }
}
```

Benefits: - No if-else chain - Polymorphism handles behavior - Easy to extend - Follows Single Responsibility Principle

## 4. Key Clean Code Concepts

### 1. Single Responsibility Principle

Each class should do one thing.

### 2. Open/Closed Principle

Open for extension, closed for modification.

### 3. Encapsulation

Hide complexity inside classes.

#### 4. Polymorphism

Replace conditionals with object behavior.

#### 5. Polymorphism Example

```
public interface ArgumentMarshaler {
    void set(Iterator<String> currentArgument) throws ArgsException;
}

public class IntegerArgumentMarshaler implements ArgumentMarshaler {
    private int intValue;

    public void set(Iterator<String> currentArgument) throws ArgsException {
        String param = currentArgument.next();
        intValue = Integer.parseInt(param);
    }
}
```

#### 6. Refactoring Insight

Clean code is not written in one go. It is achieved through multiple iterations: - Write messy code first - Refactor gradually - Keep system working (TDD) - Improve structure step by step

#### 7. Quotes

*"Clean code always looks like it was written by someone who cares." – Robert C. Martin*

*"Programming is the art of writing code that humans can understand."*

*"Messy code works. Clean code lasts."*

#### 8. Conclusion

The Args case study shows that scalability breaks bad design. Only through refactoring, abstraction, and discipline can code remain maintainable.